

UNIVERSIDADE FEDERAL DE MINAS GERAIS - CAMPUS PAMPULHA

Ana Paula Amorim Almeida

Pedro Lucas Gomes Sousa

Vanessa Nascimento Santos

**Relatório do Trabalho Final do Laboratório de Sistemas Digitais:
Elevador**

**Belo Horizonte
2022**

I – INTRODUÇÃO

Este trabalho teve como objetivo colocar em prática os conhecimentos adquiridos durante as aulas do Laboratório de Sistemas Digitais além de explorar a interação do FPGA e do kit Altera DE2 com o mundo físico.

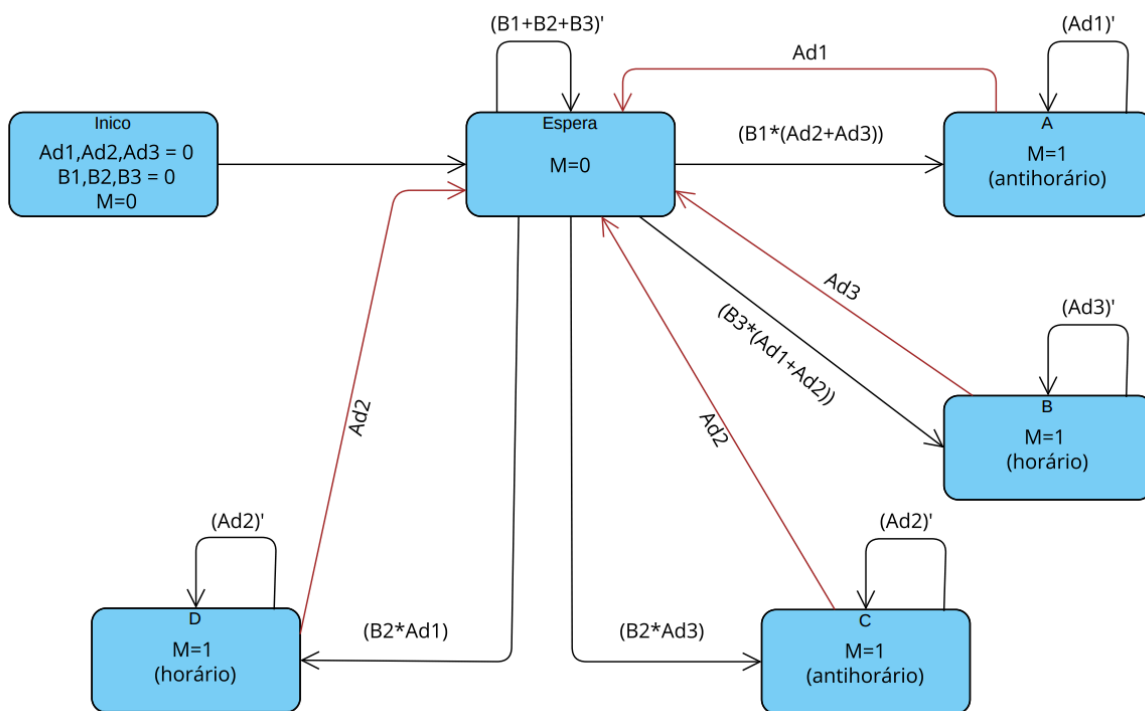
Foi desenvolvido o projeto de um elevador de papelão de 3 andares controlado pelo FPGA e por um Arduino Due. O código voltado para o FPGA foi escrito em VHDL a partir do software Quartus II, já o código do Arduino foi escrito em C++ utilizando o Arduino IDE. A comunicação entre eles foi realizada a partir das portas GPIO do kit Altera DE2.

O grupo fez o uso das monitorias para resolver problemas encontrados no código em VHDL, além de dimensionar e testar o circuito elétrico para ligar e desligar o motor do elevador.

II – PROCEDIMENTOS

A. Construção da Máquina de Estados Finitos

O primeiro passo tomado pelo grupo foi a criação de uma FSM. A máquina de estados finitos foi responsável por guiar a escrita do código através da separação dos estados e das condições de transição, além do que acontece dentro de cada estado.



Ad1, Ad2 e Ad3 referem-se aos andares de forma que possuem o valor '1' quando o elevador se encontra naquele respectivo andar. B1, B2 e B3 referem-se aos botões de seleção dos andares e possuem o valor '1' quando um andar é selecionado. M refere-se ao motor, M = 1 significa motor ligado sendo que o sentido pode ser horário ou anti horário, nessa ordem, equivalente ao elevador subir ou descer.

B. Código em VHDL

Essa etapa consistiu em transformar a FSM descrita anteriormente em um código VHDL (na última página deste relatório, anexamos o RTL Viewer associado ao circuito digital resultante do FPGA). Para a realização do código iniciamos definindo as entradas e as saídas e o tipo de cada uma. Além disso, foram definidos os estados/casos da FSM:

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity fsm_elevador is
port(
    andar1 : in std_logic; --recebe do arduino
    andar2 : in std_logic; --recebe do arduino
    andar3 : in std_logic; --recebe do arduino
    botao1 : in std_logic; --chave switch
    botao2 : in std_logic; --chave switch
    botao3 : in std_logic; --chave switch
    b1      : out std_logic; --manda para o arduino
    b2      : out std_logic; --manda para o arduino
    b3      : out std_logic; --manda para o arduino
    reset   : in std_logic;
    motor   : out std_logic; --manda para o arduino
    sentido : out std_logic --manda para o arduino
);
end fsm_elevador;
```

```
architecture hardware of fsm_elevador is
    type casos is (espera, casoA, casoB, casoC, casoD);
    signal aux1, aux2, aux3 : std_logic; --salva o estado ('0' ou '1') do botao
    signal estado : casos;
```

Foram criados 3 Process. O “my_process1” para salvar o estado dos botões nas variáveis auxiliares, que é útil para que não seja preciso pressionar o botão até que o elevador chegue no andar desejado (apesar de termos usado as chaves switch do kit Altera DE2, push buttons seriam ideais para essa aplicação). No “my_process2”, usamos sentenças case-when e if-elsif com o objetivo de determinar as condições para as trocas de estados. Já o “my_process3” tem o intuito de determinar o que acontece dentro de cada estado, como, por exemplo, quando o motor deve ser acionado:

```
begin
    b1 <= botao1;
    b2 <= botao2;
    b3 <= botao3;

    my_process1 : process(botao1, botao2, botao3, andar1, andar2, andar3)
    begin
        if (botao1 = '1') then
```

```

aux1 <= botao1;
elsif(botao2 = '1') then
aux2 <= botao2;
elsif(botao3 = '1') then
aux3 <= botao3;
end if;

if(andar1 = '1') then
aux1 <= '0';
elsif(andar2 = '1') then
aux2 <= '0';
elsif(andar3 = '1') then
aux3 <= '0';
end if;

end process my_process1;

my_process2 : process(andar1, andar2, andar3, aux1, aux2, aux3)
begin

if ((aux1 OR aux2 OR aux3) = '0') then
    estado <= espera;

elsif((aux1 OR aux2 OR aux3) = '1') then
    case estado is
        when espera =>
            if(aux1 ='1' AND ((andar2 ='1') OR (andar3 ='1' ))) then
                estado <= casoA;

                elsif(aux3 ='1' AND ((andar1 ='1') OR (andar2 ='1'))))then
                    estado <= casoB;

                elsif((aux2 AND andar3) = '1' )then
                    estado <= casoC;

                elsif((aux2 AND andar1) = '1')then
                    estado <= casoD;

                else
                    estado <= espera;

            end if;
        when casoA =>
            if(andar1 = '0')then
                estado <= casoA;
            else
                estado <= espera;

            end if;

        when casoB =>
            if(andar3 = '0')then
                estado <= casoB;
            else

                estado <= espera;

            end if;

        when casoC=>
            if(andar2 = '0')then
                estado <= casoC;

```

```

        else

            estado <= espera;

        end if;

    when casoD =>
        if(andar2 = '0')then
            estado <= casoD;
        else

            estado <= espera;

        end if;

    end case;
end if;
end process my_process2;

my_process3 : process(estado)
begin

    case estado is
        when espera =>
            sentido <= '0';
            motor<= '0';

        when casoA =>
            sentido <= '0';
            motor <= '1';

        when casoB =>
            sentido <= '1';
            motor <= '1';

        when casoC=>
            sentido <= '0';
            motor <= '1';

        when casoD =>
            sentido <= '1';
            motor <= '1';

    end case;
end process my_process3;
end hardware;

```

C. Código em C++

O microcontrolador Arduino Due foi utilizado com o objetivo de enviar o sinal para o motor, além de, por meio de um sensor ultrassônico, receber a informação de distância e, dessa forma, transmitir ao FPGA em qual andar o elevador se encontra com sinais binários.

Para possibilitar que o motor girasse em ambos os sentidos (horário e anti-horário) foi montada uma Ponte H com auxílio de transistores BC337 funcionando em sua região de saturação, ou seja, como chaves. Quando o arduino envia um sinal para a Ponte H, ele satura um par de transistores, permitindo o acionamento do motor em um sentido, o outro par de transistores ativa o motor no outro sentido. O circuito foi montado em duas protoboards e foram utilizados diversos jumpers.

O arduino, além de enviar informações para o FPGA, recebe a informação/sinal de qual botão foi pressionado, o sentido de rotação do motor e se é ou não para ligar o motor. As variáveis que representam os sinais enviados pelo FPGA são: liga_motor, sentido_rot, botao1, botao2 e botao3. As entradas e saídas utilizadas no arduino foram omitidas para simplificar a visualização do código.

Optamos em utilizar o Arduino Due pois o seu sinal de saída “HIGH” equivale a 3.3V, compatível com o sinal ‘1’ das portas GPIO do FPGA e, como esperado, o sinal “LOW” equivale ao ‘0’ do FPGA.

```
void setup()
{
  Serial.begin(9600);
  pinMode(horario, OUTPUT);
  pinMode(anti, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(liga_motor, INPUT);
  pinMode(sentido_rot, INPUT);
  pinMode(andar1, OUTPUT);
  pinMode(andar2, OUTPUT);
  pinMode(andar3, OUTPUT);
  pinMode(botao1, INPUT);
  pinMode(botao2, INPUT);
  pinMode(botao3, INPUT);
}

void loop()
{
  distancia = ultrasonic.Ranging(CM);

  if((digitalRead(botao1) == HIGH) && (digitalRead(liga_motor) == HIGH)){
    while(distancia > 5){
```

```

        digitalWrite(anti, HIGH);
        distancia = ultrasonic.Ranging(CM);
    }
    digitalWrite(anti, LOW);
    digitalWrite(andar1, HIGH);
    digitalWrite(andar2, LOW);
    digitalWrite(andar3, LOW);
}

else if ((digitalRead(botao2) == HIGH) && (digitalRead(liga_motor) == HIGH)){
    if(digitalRead(sentido_rot) == HIGH){
        while(distancia < 10){
            digitalWrite(horario, HIGH);
        }
        digitalWrite(horario, LOW);
        digitalWrite(andar1, LOW );
        digitalWrite(andar2, HIGH);
        digitalWrite(andar3, LOW);
    }
}

else if (digitalRead(sentido_rot) == LOW){
    while(distancia > 10){
        digitalWrite(anti, HIGH);
        distancia = ultrasonic.Ranging(CM);
    }
    digitalWrite(anti, LOW);
    digitalWrite(andar1, LOW );
    digitalWrite(andar2, HIGH);
    digitalWrite(andar3, LOW);
}
}

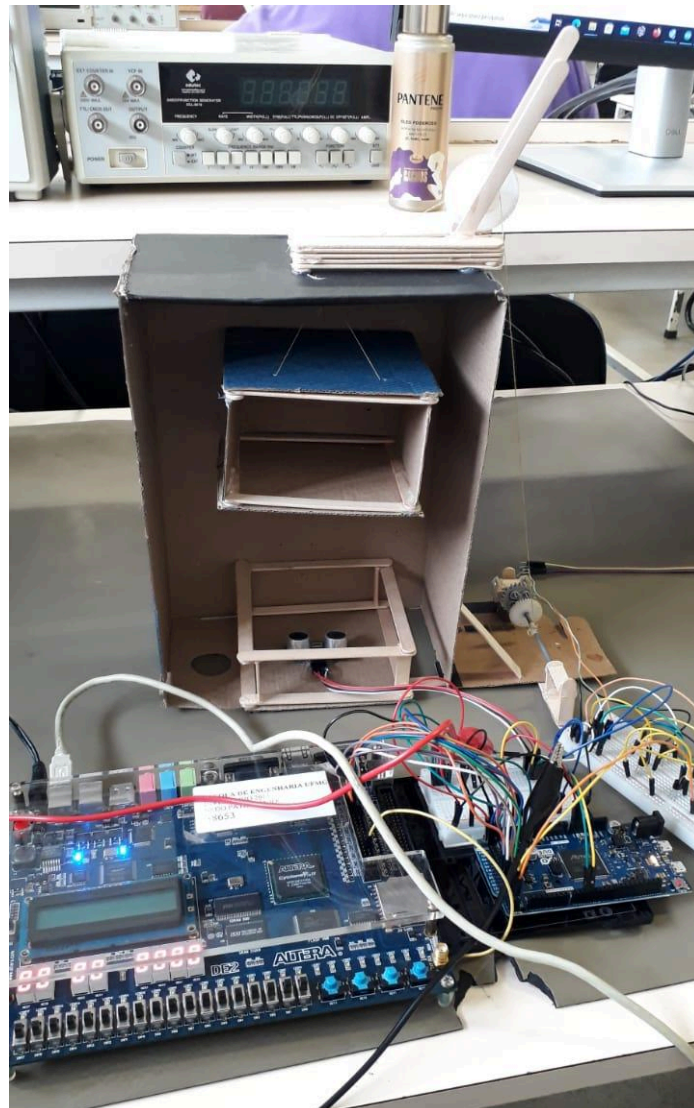
else if ((digitalRead(botao3) == HIGH) && (digitalRead(liga_motor) == HIGH)){
    while(distancia < 15){
        digitalWrite(horario, HIGH);
        distancia = ultrasonic.Ranging(CM);
    }

    digitalWrite(horario, LOW);
    digitalWrite(andar1, LOW );
    digitalWrite(andar2, LOW);
    digitalWrite(andar3, HIGH);
}
}
}

```


D. Montagem da parte física

Para montar a parte física do elevador utilizamos uma caixa de papelão, palitos de picolé, cola quente, um eixo com engrenagens de brinquedo, linha de costura e um motor cc.



III – Conclusão

O projeto funcionou conforme o esperado e permitiu que o grupo explorasse e colocasse em prática não apenas os conhecimentos em VHDL como o de outras áreas, revelando uma interdisciplinaridade no trabalho. Ademais, também permitiu a aprendizagem sobre funcionalidades do Altera DE2 que ainda não haviam sido utilizadas nas aulas, como as portas GPIO.